Introduction to Software Engineering

# Software Design

Software Engineering Department
Faculty of Information and Technology
University of Science

# Table of Contents


**1 Member Contribution Assessment 2**

**2 Conceptual Model 3**

**3 Architectural Design 4**

3.1 Architecture Diagram ..... 4

3.2 Class Diagram ..... 4

3.3 Class Specifications ..... 5

3.3.1 Class C1 5

**4 Data Design 6**

4.1 Data Diagram ..... 6

4.2 Data Specification ..... 6

**5 User Interface and User Experience Design 7**

5.1 Screen Diagram ..... 7

5.2 Screen Specifications ..... 7

5.2.1 Screen “A” 7

5.2.2 Screen “B” 7

# Software Design

## Objectives

This document focus on the following topics:

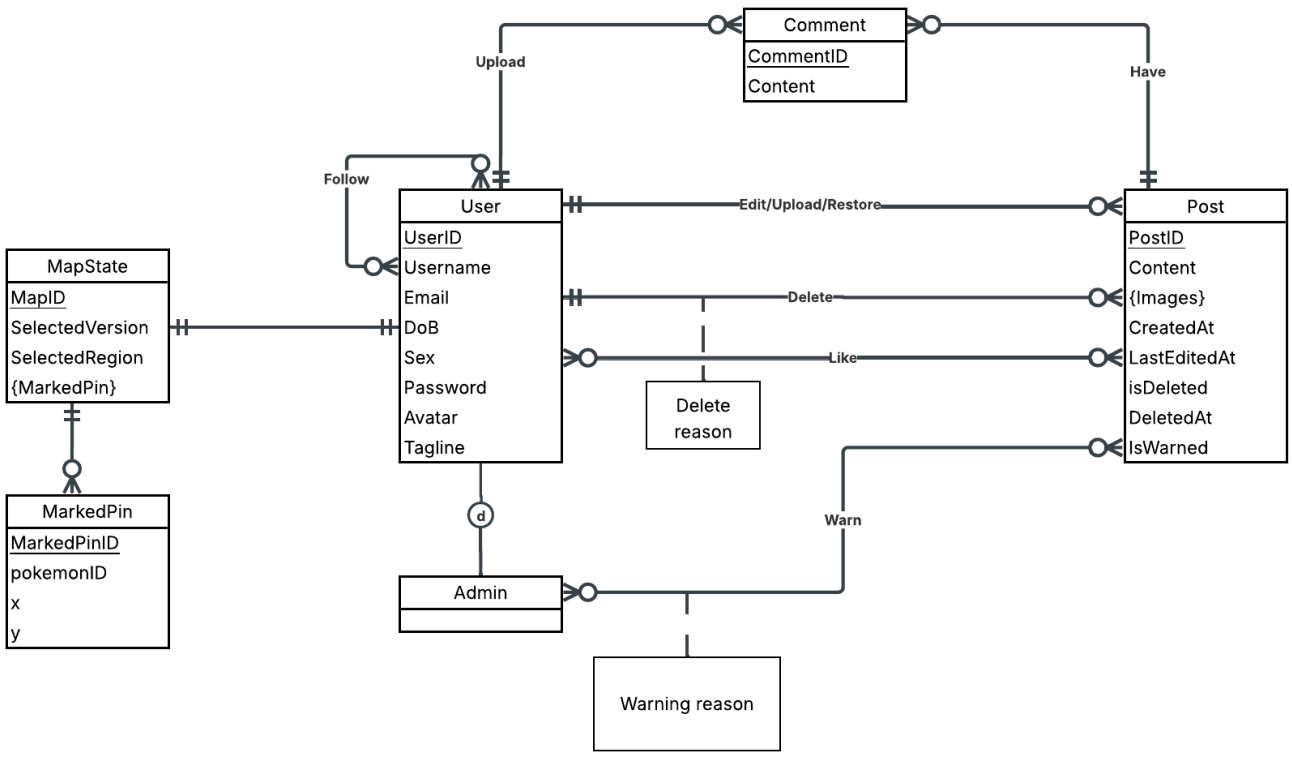
- ✔ Complete the Software Design Document with the following contents:
  - Conceptual Model
  - Architectural Design
  - Data Design
  - User Interface Design
- ✔ Understanding the Software Design Document.

# 1 Member Contribution Assessment

| ID | Name | Role | Contribution (%) | Signature |
|---|---|---|---|---|
| 23127061 | Trương Lý Khải | **Project Manager**<br>Business Analyst<br>Tester | 100% | |
| 23127207 | Đặng Đăng Khoa | Designer<br>Backend Developer | 100% | |
| 23127271 | Võ Ngọc Bích Trâm | Designer<br>Frontend Developer<br>Tester | 100% | |
| 23127404 | Lê Tuấn Lộc | Fullstack Developer & Engineer | 100% | |
| 23127486 | Phan Quốc Thịnh | Designer<br>Frontend Developer | 100% | |

# 2 Conceptual Model

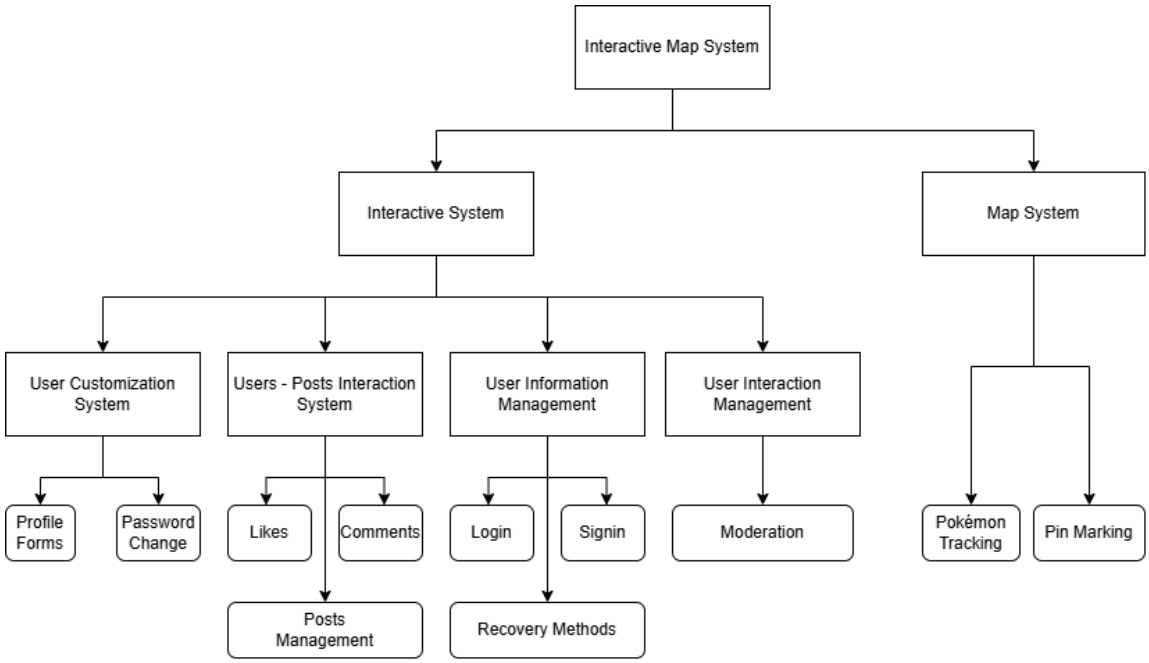
The following diagram is the ERD model for our project:



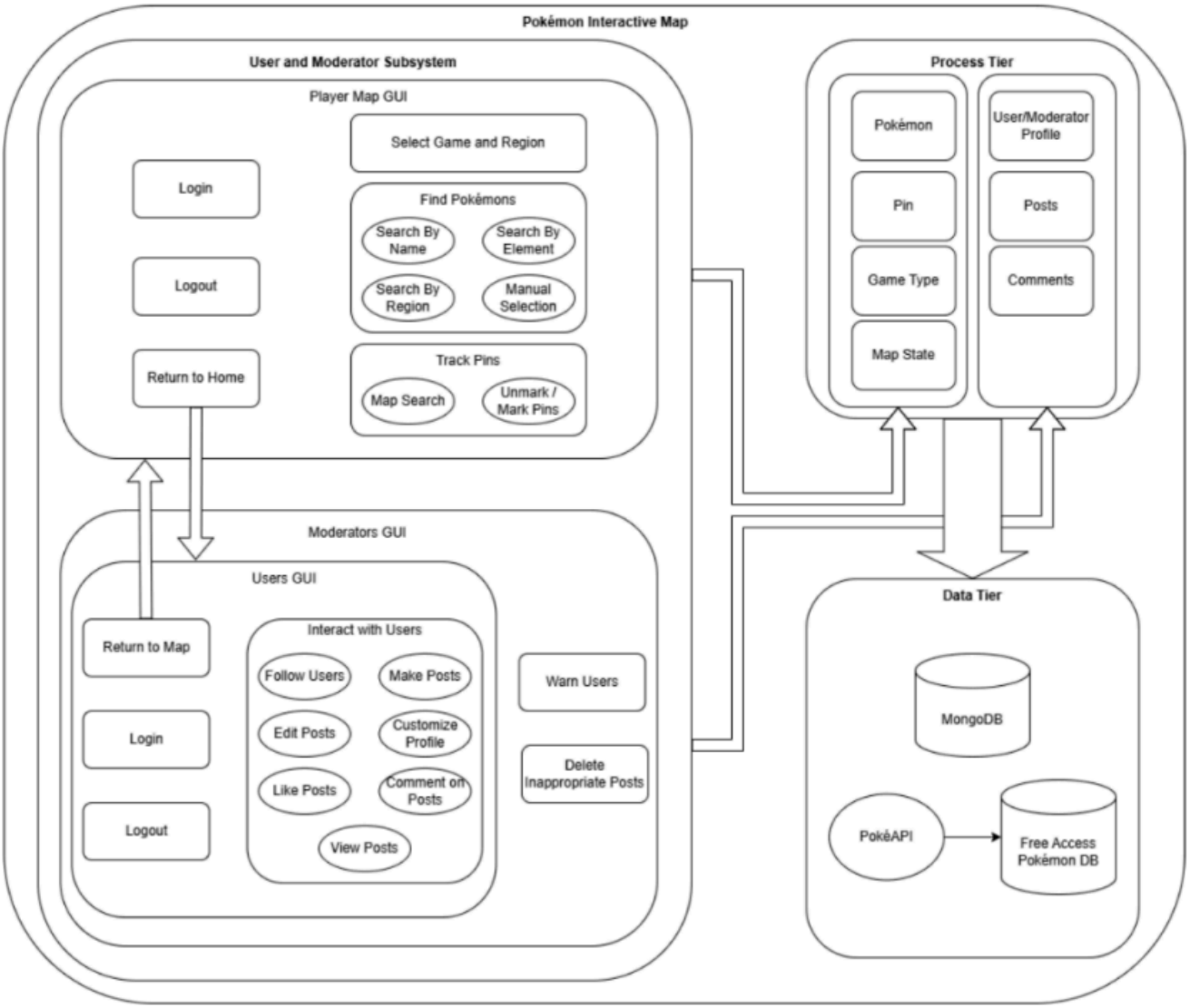
# 3 Architectural Design

## 3.1 Architecture Diagram

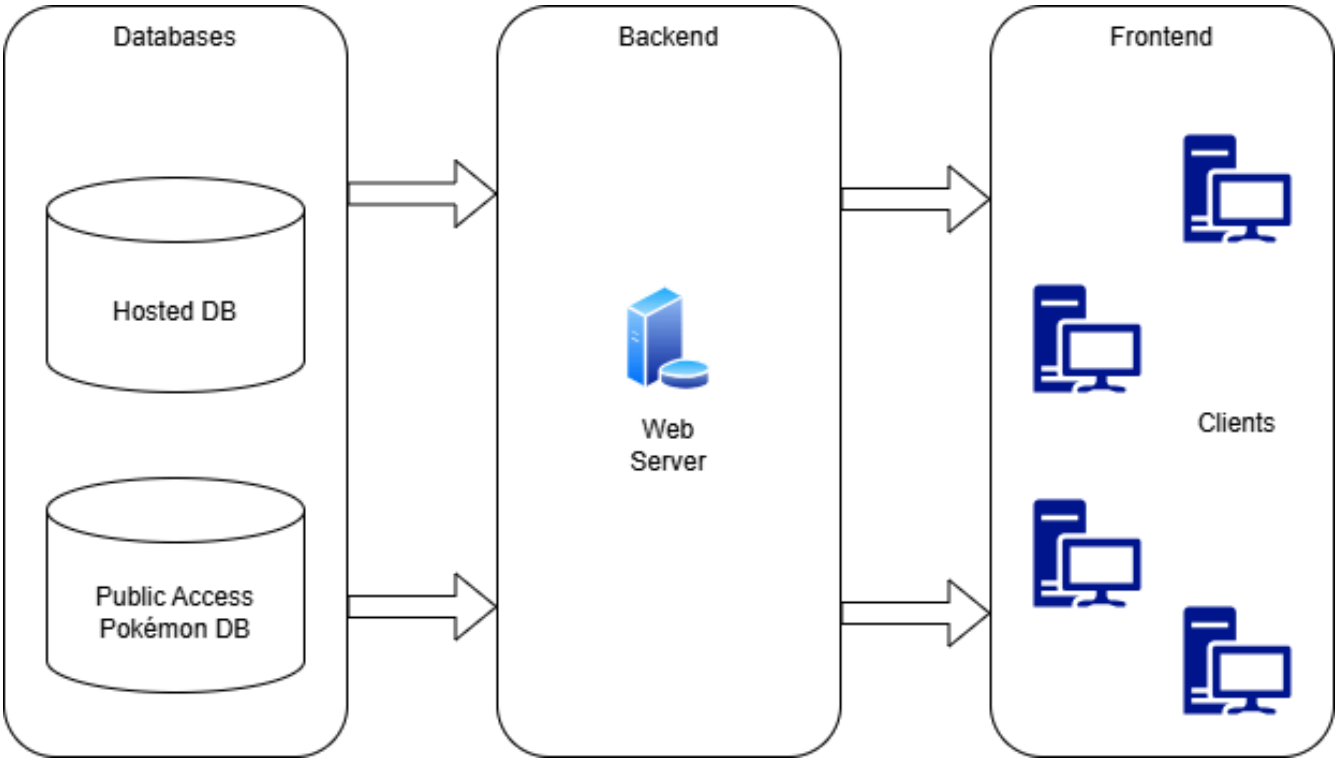
The following diagram is a system decomposition tree mainly focusing around **systems and functions**:



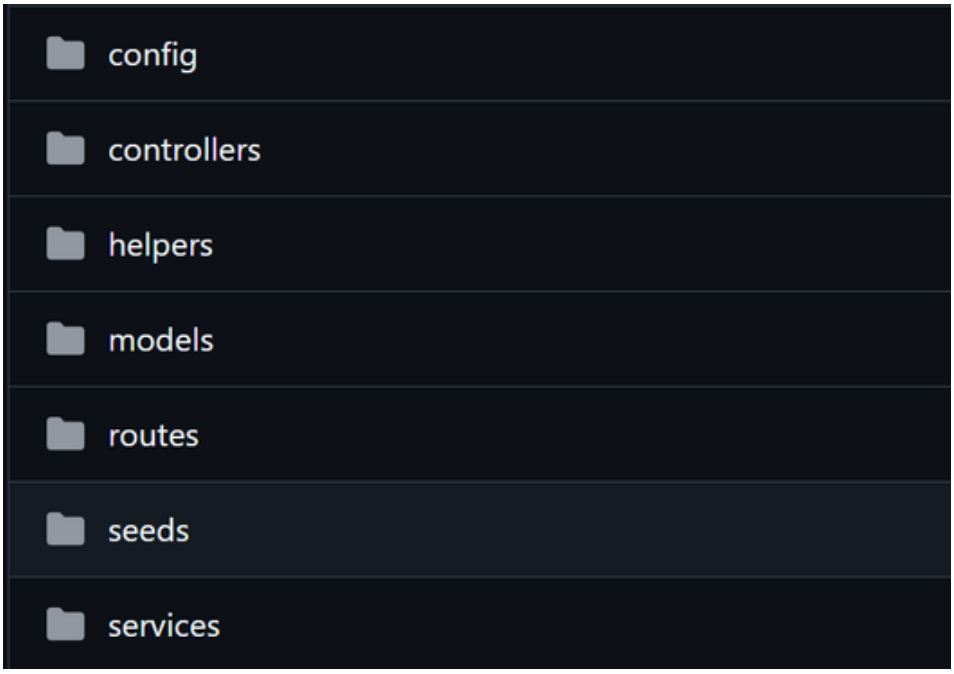

The following diagram is the original architectural diagram of the entire system mainly focusing around **use cases and data sets**:


Pokémon Interactive Map
User and Moderator Subsystem
Player Map GUI
Select Game and Region
Login
Find Pokémons
Search By Name
Search By Element
Logout
Search By Region
Manual Selection
Return to Home
Track Pins
Map Search
Unmark / Mark Pins
Moderators GUI
Users GUI
Return to Map
Interact with Users
Follow Users
Make Posts
Warn Users
Login
Edit Posts
Customize Profile
Delete Inappropriate Posts
Like Posts
Comment on Posts
Logout
View Posts
Process Tier
Pokémon
User/Moderator Profile
Pin
Posts
Game Type
Comments
Map State
Data Tier
MongoDB
PokéAPI
Free Access Pokémon DB

Our system also uses the MVC (Model-View-Controller) model to handle the website development, the model is as follows:

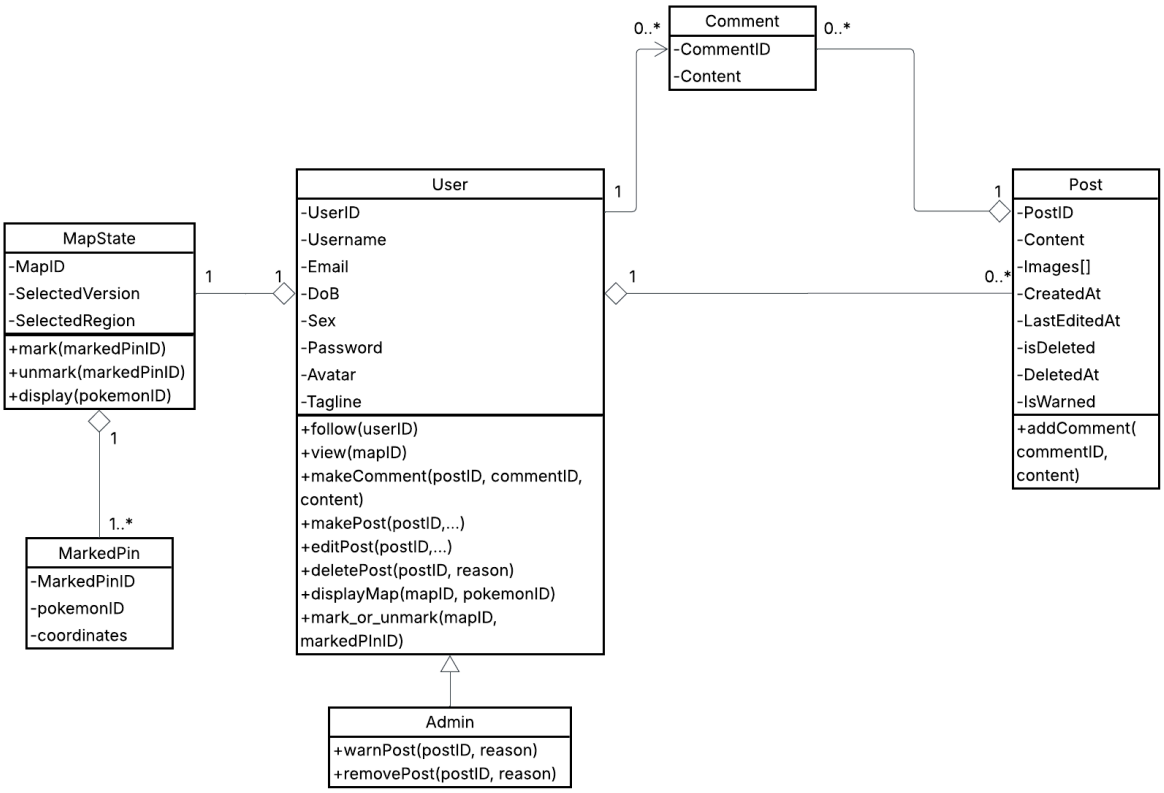



Here is the segmentation of our source code to implement the MVC model:



## 3.2 Class Diagram

Through the ERD, we transform it into a class diagram:



## 3.3 Class Specifications

### 3.3.1 Class User

Attributes

| Seq | Property | Modifier | Constraint | Description |
|---|---|---|---|---|
| 1 | **UserID** | Private | Primary Key | A unique identifier for the User. |
| 2 | **Username** | Private | Mandatory | The user's displayed name. |
| 3 | **Email** | Private | Mandatory, Validated Email | The user's email address. |
| 4 | **DoB** | Private | Mandatory | The user’s date of birth. |
| 5 | **Sex** | Private | Optional | The user's sex. |
| 6 | **Password** | Private | Mandatory, Hashed | The user's password for authentication. |
| 7 | **Avatar** | Private | Optional, Has a default option | The user's profile image. |
| 8 | **Tagline** | Private | Optional | A short descriptive phrase for the user's profile. |

Methods

| Seq | Property | Modifier | Constraint | Description |
|---|---|---|---|---|
| 1 | **follow(userID)** | Public | Requires valid userID | Allows the current user to follow another specified user. |
| 2 | **view(mapID)** | Public | Requires valid mapID | Records that the user has selected to view the map. |
| 3 | **makeComment(postID, commentID, content)** | Public | Requires valid postID and commentID | Creates a new comment on a specified post with the given content. |
| 4 | **makePost(postID, ...)** | Public | Requires valid postID | Creates a new post with the given details as parameters. |
| 5 | **editPost(postID, ...)** | Public | Requires valid postID | Modifies the content or details of an existing post. |
| 6 | **deletePost(postID, reason)** | Public | Requires valid postID | Removes a user's post, with the reasoning to do so. |

| | | | | |
|---|---|---|---|---|
| 7 | **displayMap(mapID, pokemonID)** | Public | Requires valid mapID and pokemonID | Fetches and displays all corresponding Pokémon's pins on the chosen map. |
| 8 | **mark_or_unmark (mapID, markedPinID)** | Public | Requires valid mapID and markedPinID | Toggles the state of a pin on the map. |
| 9 | **toggleLike(postID)** | Public | Requires valid postID | Toggles the like state of a post. |

### 3.3.2 Class Post

Attributes

| Seq | Property | Modifier | Constraint | Description |
|---|---|---|---|---|
| 1 | **PostID** | Private | Primary Key | A unique identifier for the Post. |
| 2 | **Content** | Private | Optional, Has a default option | The main text body of the post. |
| 3 | **Images[]** | Private | Optional | An array or list of images attached to the post. |

| | | | | |
|---|---|---|---|---|
| 4 | **CreatedAt** | Private | Mandatory | The timestamp when the post was initially created. |
| 5 | **LastEdited At** | Private | Optional | The timestamp of the last time the post was modified. |
| 6 | **isDeleted** | Private | Mandatory | Flag indicating if the post has been deleted by the user. |
| 7 | **DeletedAt** | Private | Optional | The timestamp when the post was deleted |
| 8 | **IsWarned** | Private | Mandatory | Flag indicating if the post has received a warning from an Admin/Moderator. |
| 9 | **likeCount** | Private | Mandatory, Positive Value | The amount of likes on the post. |

Methods

| Seq | Property | Modifier | Constraint | Description |
|---|---|---|---|---|
| 1 | **addComment (commentID, content)** | Public | Requires valid commentID | Adds a new comment to a specific post. |

| | | | | |
|---|---|---|---|---|
| 2 | **displayComments()** | Public | - | Shows all comments available on the post. |

### 3.3.3 Class MapState

Attributes

| Seq | Property | Modifier | Constraint | Description |
|---|---|---|---|---|
| 1 | **MapID** | Private | Primary Key | A unique identifier for the specific state or version of the map. |
| 2 | **SelectedVersion** | Private | Optional | Indicator of the version of the game that is used. |
| 3 | **SelectedRegion** | Private | Optional | Indicator of the version of the region that is used. |

Methods

| Seq | Property | Modifier | Constraint | Description |
|---|---|---|---|---|
| 1 | **mark(markedPinID)** | Public | Requires valid markedPinID | Marks a specified marked pin to the current map state. |

| | | | | |
|---|---|---|---|---|
| 2 | **unmark(markedPinID)** | Public | Requires valid markedPinID | Unmarks a specified marked pin from the current map state. |
| 3 | **display(pokemonID)** | Public | Requires valid pokemonID | Renders all corresponding pins and their states of a specific Pokémon. |

### 3.3.4 Class MarkedPin

Attributes

| Seq | Property | Modifier | Constraint | Description |
|---|---|---|---|---|
| 1 | **MarkedPinID** | Private | Primary Key | A unique identifier for the specific pin on the map. |
| 2 | **pokemonID** | Private | Mandatory, Valid | The type of Pokémon that the pin is referring to. |
| 3 | **coordinates** | Private | Mandatory, Both Values Positive | Coordinates of the pin |
| 4 | **status** | Private | Mandatory, Default Value 0 | Boolean status of the pin to indicate the mark |

### 3.3.5 Class Admin

Attributes

| Seq | Property | Modifier | Constraint | Description |
|---|---|---|---|---|
| 1 | **UserID** | Private | Primary Key, Inherited From User | A unique identifier for the User. |
| 2 | **Username** | Private | Mandatory, Inherited From User | The user's displayed name. |
| 3 | **Email** | Private | Mandatory, Validated Email, Inherited From User | The user's email address. |
| 4 | **DoB** | Private | Mandatory, Inherited From User | The user’s date of birth. |
| 5 | **Sex** | Private | Optional, Inherited From User | The user's sex. |
| 6 | **Password** | Private | Mandatory, Hashed, | The user's password for authentication. |

| | | | | |
|---|---|---|---|---|
| | | | Inherited From User | |
| 7 | **Avatar** | Private | Optional, Has a default option, Inherited From User | The user's profile image. |
| 8 | **Tagline** | Private | Optional, Inherited From User | A short descriptive phrase for the user's profile. |

Methods

| Seq | Property | Modifier | Constraint | Description |
|---|---|---|---|---|
| 1 | **follow(userID)** | Public | Requires valid userID, Inherited From User | Allows the current user to follow another specified user. |
| 2 | **view(mapID)** | Public | Requires valid mapID, Inherited From User | Records that the user has selected to view the map. |
| 3 | **makeComment(postID, commentID, content)** | Public | Requires valid postID and commentID, Inherited From User | Creates a new comment on a specified post with the given content. |

| | | | | |
|---|---|---|---|---|
| 4 | **makePost(postID , ...)** | Public | Requires valid postID, Inherited From User | Creates a new post with the given details as parameters. |
| 5 | **editPost(postID, ...)** | Public | Requires valid postID, Inherited From User | Modifies the content or details of an existing post. |
| 6 | **deletePost(postID, reason)** | Public | Requires valid postID, Inherited From User | Removes a user's post, with the reasoning to do so. |
| 7 | **displayMap(mapID, pokemonID)** | Public | Requires valid mapID and pokemonID, Inherited From User | Fetches and displays all corresponding Pokémon's pins on the chosen map. |
| 8 | **mark_or_unmark (mapID, markedPinID)** | Public | Requires valid mapID and markedPinID, Inherited From User | Toggles the state of a pin on the map. |
| 9 | **toggleLike(postID)** | Public | Requires valid postID, Inherited From User | Toggles the like state of a post. |

| | | | | |
|---|---|---|---|---|
| 10 | **warnPost(postID, reason)** | Public | Requires valid postID | Warns a post of another user, with a specific reasoning given. |
| 11 | **removePost(post ID, reason)** | Public | Requires valid postID | Removes a post of another user, with a specific reasoning given. |

### 3.3.6 Abstract Classes

Attributes of Comment:

| Seq | Property | Modifier | Constraint | Description |
|---|---|---|---|---|
| 1 | **Content** | Private | - | The content of a comment of a post from a user. |

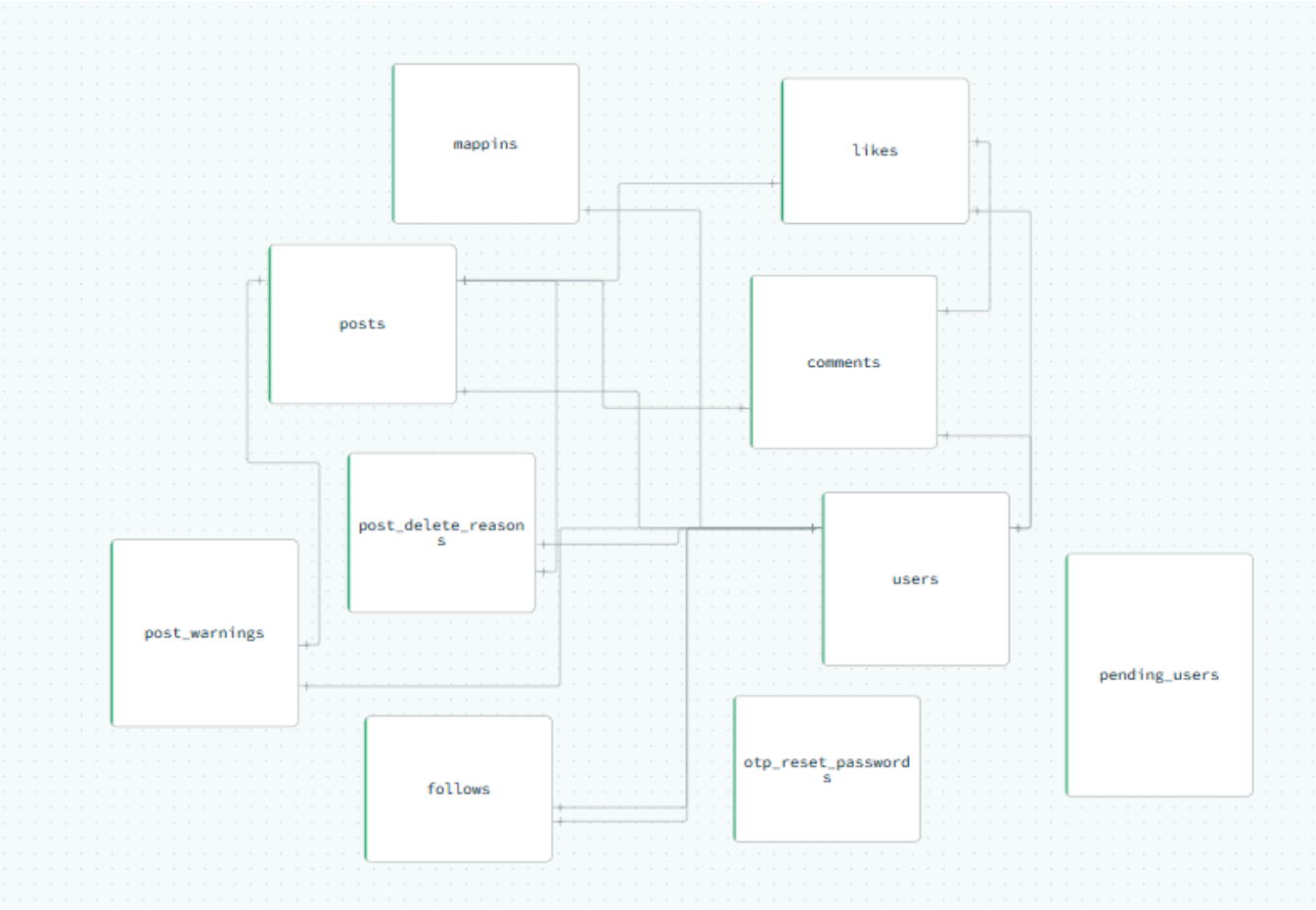
Attributes of Like:

| Seq | Property | Modifier | Constraint | Description |
|---|---|---|---|---|
| 1 | **IsLiked** | Private | - | Indicator of the like status of a post from a user. |

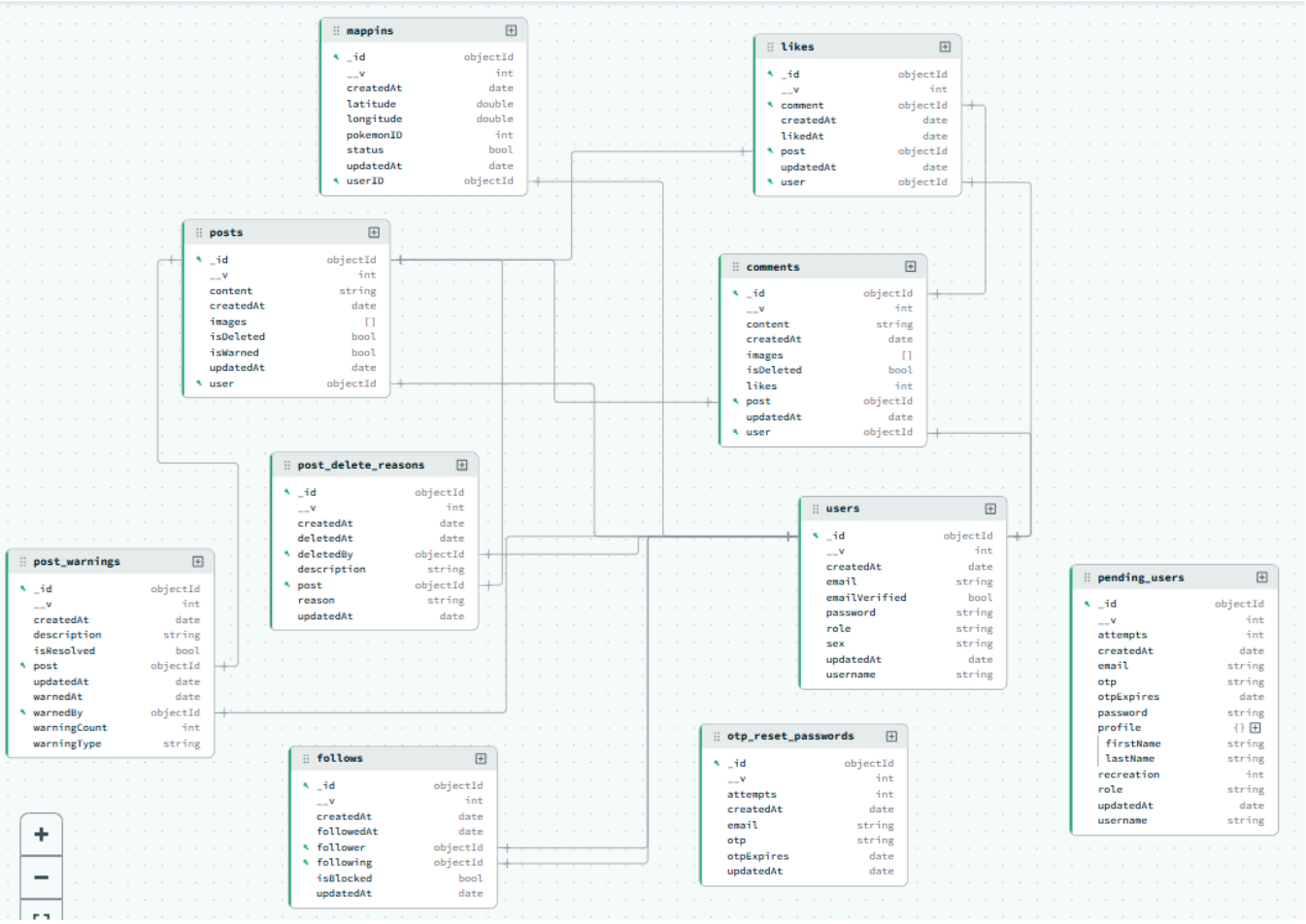
# 4 Data Design

## 4.1 Data Diagram

Here is the current simplified relationship diagram of our project:




Here is a more detailed diagram:




## 4.2 Data Specification

Every individual data component is built within MongoDB, which can display the relationship diagram along with the data type specification of each table.

There are 8 main tables storing data that is permanent with intricate relationships between them along with 2 more side tables ("otp_reset_passwords" and "pending_users") to store temporary data used for purposes with no maintenance requirements.

Both side tables have an indirect relationship with the "users" table where its temporary data can affect permanent data within "users". No foreign keys are needed as data is deleted immediately after use and corresponding data is modified.

### *4.2.1. Users Table*

- Data List and Relationship List:

**users**

| | |
|---|---|
| _id | objectId |
| __v | int |
| createdAt | date |
| email | string |
| emailVerified | bool |
| password | string |
| username | string |

RELATIONSHIPS 7 ADD RELATIONSHIP

- comments.user → users._id
- follows.follower → users._id
- follows.following → users._id
- likes.user → users._id
- post_delete_reasons.deletedBy → users._id
- post_warnings.warnedBy → users._id
- posts.user → users._id

- Constraints include only the primary key.
- Index Partitioning used:

| Name & Definition | Type | Size | Usage | Properties | Status |
|---|---|---|---|---|---|
| _id_ | REGULAR | 36.9 kB | 93 (since Fri Nov 21 2025) | UNIQUE | READY |
| email_1 | REGULAR | 36.9 kB | 187 (since Fri Nov 21 2025) | UNIQUE | READY |
| username_1 | REGULAR | 36.9 kB | 0 (since Fri Nov 21 2025) | UNIQUE | READY |

- Data Example:

```
_id: ObjectId('6920417ec98e6e083d55fba3')
email : "letuanloc1412@gmail.com"
password : "$2b$10$ZVAWWGHeMUNGqw7zF/K2WujciNX1wI2ujRE5AOo9tCoKvJM5TVnkK"
username : "Lokdeptrai"
emailVerified : true
createdAt : 2025-11-21T10:39:58.080+00:00
__v : 0
```

### *4.2.2. Posts Table*

- Data List and Relationship List:

**posts**

| | |
|---|---|
| _id | objectId |
| __v | int |
| content | string |
| createdAt | date |
| images | [] |
| isDeleted | bool |
| isWarned | bool |
| updatedAt | date |
| user | objectId |

**RELATIONSHIPS** 5 — ADD RELATIONSHIP

- comments.post → posts._id
- likes.post → posts._id
- post_delete_reasons.post → posts._id
- post_warnings.post → posts._id
- posts.user → users._id

- Constraints include only the primary key and foreign key.
- Index Partitioning used:

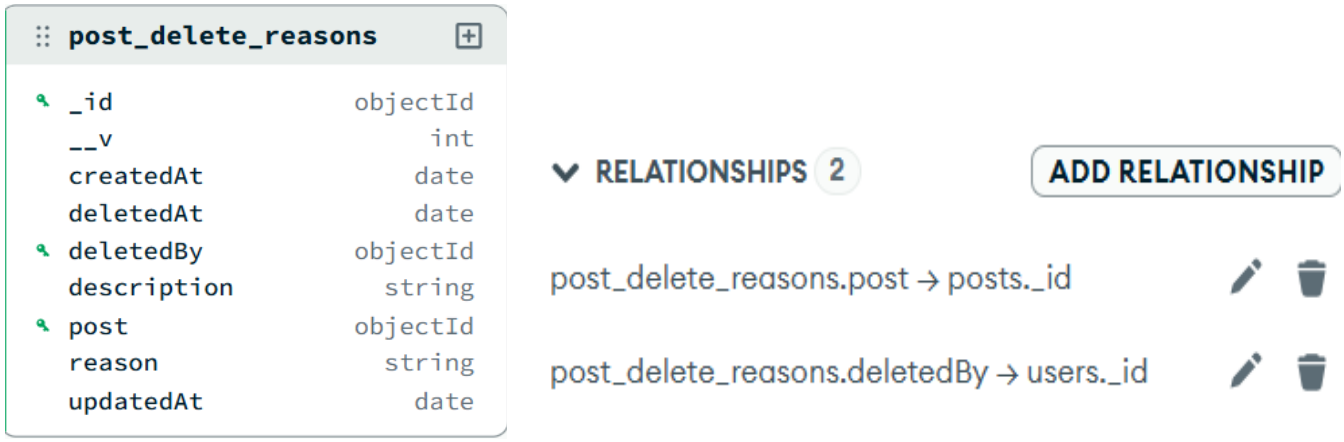
| Name & Definition | Type | Size | Usage | Properties | Status |
|---|---|---|---|---|---|
| _id_ | REGULAR | 32.8 kB | 4 (since Tue Dec 02 2025) | UNIQUE | READY |

- Data Example:

```
_id: ObjectId('692ff92ef6ac9f963a20d0e9')
user : ObjectId('692ff8182b1a1270d2e04a1d')
content : "Just caught an amazing Pikachu at the park! 🎉 Anyone else playing?"
▸ images : Array (1)
createdAt : 2025-12-02T08:47:42.178+00:00
isDeleted : false
isWarned : false
__v : 0
updatedAt : 2025-12-03T08:47:42.201+00:00
```

### *4.2.3. Deleted Posts' Reasons Table*

- Data List and Relationship List:




- Constraints include only the primary key and foreign keys.
- Index Partitioning used:

| Name & Definition | Type | Size | Usage | Properties | Status |
|---|---|---|---|---|---|
| _id_ | REGULAR | 36.9 kB | 6 (since Tue Dec 02 2025) | UNIQUE | READY |

- Data Example:

```
_id: ObjectId('692ff92ef6ac9f963a20d103')
post : ObjectId('692ff92ef6ac9f963a20d0e9')
reason : "spam"
description : "Duplicate post content"
deletedBy : ObjectId('692ff8182b1a1270d2e04a1d')
deletedAt : 2025-12-03T08:47:42.597+00:00
__v : 0
createdAt : 2025-12-03T08:47:42.598+00:00
updatedAt : 2025-12-03T08:47:42.598+00:00
```

### *4.2.4. Posts' Warnings Table*

- Data List and Relationship List:

**post_warnings**

| | |
|---|---|
| _id | objectId |
| __v | int |
| createdAt | date |
| description | string |
| isResolved | bool |
| post | objectId |
| updatedAt | date |
| warnedAt | date |
| warnedBy | objectId |
| warningCount | int |
| warningType | string |

RELATIONSHIPS 2 ADD RELATIONSHIP

post_warnings.post → posts._id

post_warnings.warnedBy → users._id

- Constraints include only the primary key and foreign keys.
- Index Partitioning used:

| Name & Definition | Type | Size | Usage | Properties | Status |
|---|---|---|---|---|---|
| _id_ | REGULAR | 24.6 kB | 9 (since Tue Dec 02 2025) | UNIQUE | READY |

- Data Example:

```
_id: ObjectId('692ff92ef6ac9f963a20d105')
post : ObjectId('692ff92ef6ac9f963a20d0ea')
warningType : "misleading"
description : "Post contains unverified trading information"
warningCount : 1
warnedBy : ObjectId('692ff8182b1a1270d2e04a1d')
warnedAt : 2025-12-03T00:47:42.697+00:00
isResolved : false
__v : 0
createdAt : 2025-12-03T08:47:42.698+00:00
updatedAt : 2025-12-03T08:47:42.698+00:00
```

### *4.2.5. Follows Table*

- Data List and Relationship List:

**follows**

| | |
|---|---|
| _id | objectId |
| __v | int |
| createdAt | date |
| followedAt | date |
| follower | objectId |
| following | objectId |
| isBlocked | bool |
| updatedAt | date |

RELATIONSHIPS 2 ADD RELATIONSHIP

follows.follower → users._id

follows.following → users._id

- Constraints include only the primary key and foreign keys.
- Index Partitioning used:

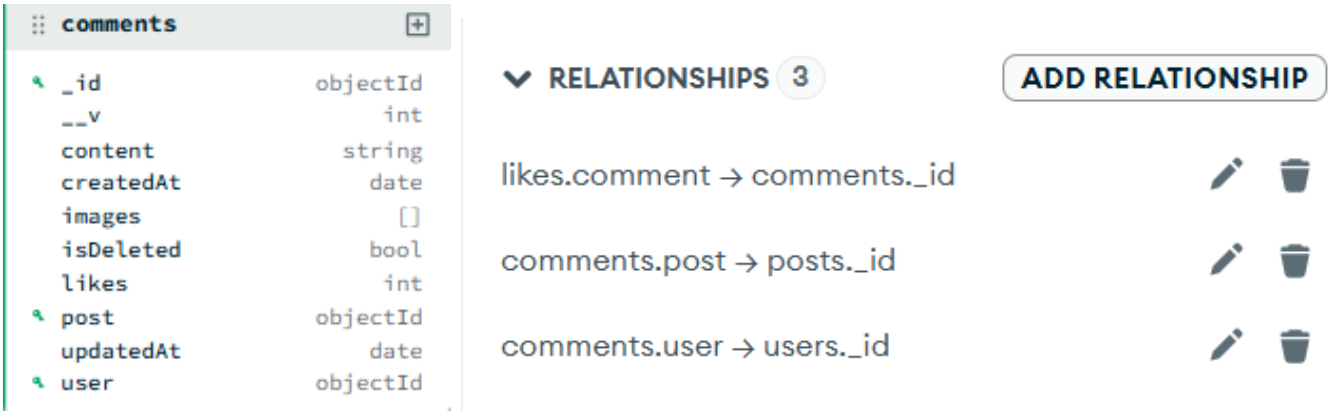
| Name & Definition | Type | Size | Usage | Properties | Status |
|---|---|---|---|---|---|
| _id_ | REGULAR | 36.9 kB | 9 (since Tue Dec 02 2025) | UNIQUE | READY |

- Data Example:

```
_id: ObjectId('692ff92ef6ac9f963a20d0f4')
follower : ObjectId('692ff9215b3e0afb5c909362')
following : ObjectId('692ff8182b1a1270d2e04a1d')
followedAt : 2025-11-03T08:47:42.399+00:00
isBlocked : false
__v : 0
createdAt : 2025-12-03T08:47:42.401+00:00
updatedAt : 2025-12-03T08:47:42.401+00:00
```

### *4.2.6. Comments Table*

- Data List and Relationship List:




- Constraints include only the primary key and foreign keys.
- Index Partitioning used:

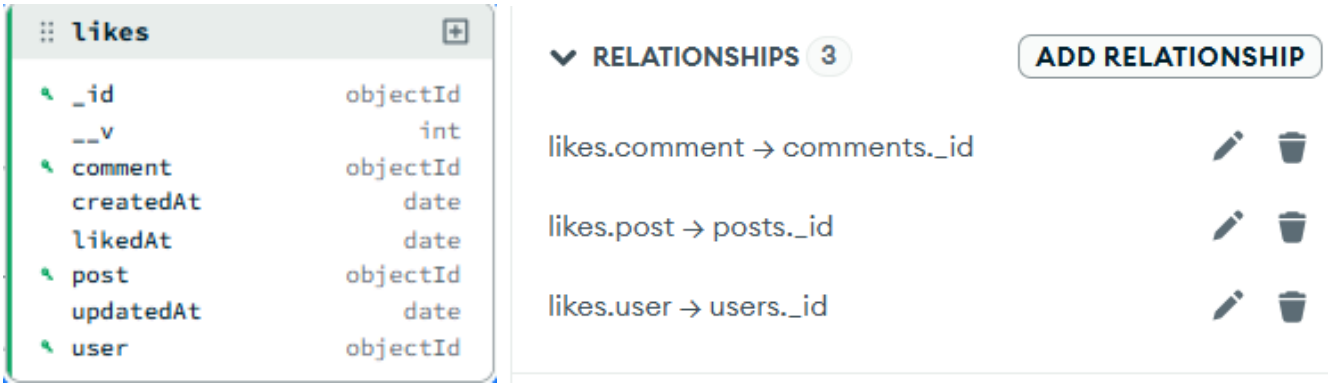
| Name & Definition | Type | Size | Usage | Properties | Status |
|---|---|---|---|---|---|
| _id_ | REGULAR | 36.9 kB | 4 (since Tue Dec 02 2025) | UNIQUE | READY |

- Data Example:

```
_id: ObjectId('692e4a7dde0664027eab6ef8')
post : ObjectId('692e4a7dde0664027eab6ef3')
user : ObjectId('692178bac7ad4093437983c7')
content : "That's awesome! Which park? I want to go there too!"
▸ images : Array (empty)
likes : 3
isDeleted : false
createdAt : 2025-12-01T06:10:05.761+00:00
__v : 0
updatedAt : 2025-12-02T02:10:05.763+00:00
```

### *4.2.7. Likes Table*

- Data List and Relationship List:




- Constraints include only the primary key and foreign keys.
- Index Partitioning used:

| Name & Definition | Type | Size | Usage | Properties | Status |
|---|---|---|---|---|---|
| _id_ | REGULAR | 36.9 kB | 7 (since Tue Dec 02 2025) | UNIQUE | READY |

- Data Example:

```
_id: ObjectId('692ff92ef6ac9f963a20d0fa')
user : ObjectId('692ff9215b3e0afb5c909362')
post : ObjectId('692ff92ef6ac9f963a20d0e9')
likedAt : 2025-12-02T10:47:42.500+00:00
__v : 0
createdAt : 2025-12-03T08:47:42.501+00:00
updatedAt : 2025-12-03T08:47:42.501+00:00
```

### *4.2.8. Mappins Table*

- Data List and Relationship List:

**mappins**

| | |
|---|---|
| _id | objectId |
| __v | int |
| createdAt | date |
| latitude | double |
| longitude | double |
| pokemonID | int |
| status | bool |
| updatedAt | date |
| userID | objectId |

RELATIONSHIPS 1 ADD RELATIONSHIP

mappins.userID → users._id

- Constraints include only the primary key and foreign key.
- Index Partitioning used:

| Name & Definition | Type | Size | Usage | Properties | Status |
|---|---|---|---|---|---|
| _id_ | REGULAR | 36.9 kB | 7 (since Wed Dec 03 2025) | UNIQUE | READY |

- Data Example:

```
_id: ObjectId('692ff9313beeaba750061c65')
pokemonID : 25
userID : ObjectId('692ff8182b1a1270d2e04a1d')
latitude : 10.8231
longitude : 106.683
status : true
createdAt : 2025-12-03T06:47:45.285+00:00
__v : 0
updatedAt : 2025-12-03T08:47:45.295+00:00
```

### *4.2.9. OTP Table*

- Data List

**otp_reset_passwords**

| | |
|---|---|
| _id | objectId |
| __v | int |
| attempts | int |
| createdAt | date |
| email | string |
| otp | string |
| otpExpires | date |
| updatedAt | date |

- Constraints include only the primary key.
- Index Partitioning used:

| Name & Definition | Type | Size | Usage | Properties | Status |
|---|---|---|---|---|---|
| _id_ | REGULAR | 24.6 kB | 19 (since Fri Nov 21 2025) | UNIQUE | READY |

- Data Example:

```
_id: ObjectId('692ff9313beeaba750061c74')
email : "user1@example.com"
otp : "111111"
otpExpires : 2025-12-03T08:55:45.946+00:00
attempts : 0
createdAt : 2025-12-03T08:45:45.946+00:00
__v : 0
updatedAt : 2025-12-03T08:47:45.947+00:00
```

### *4.2.10. Pending Users Table*

- Data List:

**pending_users**

| | |
|---|---|
| _id | objectId |
| __v | int |
| attempts | int |
| createdAt | date |
| email | string |
| otp | string |
| otpExpires | date |
| password | string |
| profile | {} |
| firstName | string |
| lastName | string |
| recreation | int |
| role | string |
| updatedAt | date |
| username | string |

- Constraints include only the primary key.
- Index Partitioning used:

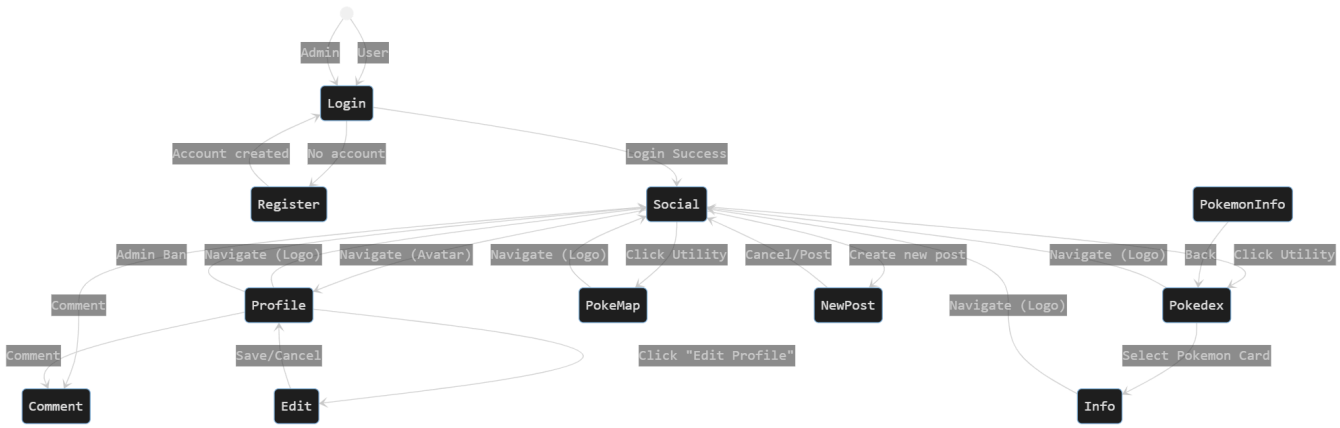
| Name & Definition | Type | Size | Usage | Properties | Status |
|---|---|---|---|---|---|
| _id_ | REGULAR | 36.9 kB | 33 (since Fri Nov 21 2025) | UNIQUE | READY |
| email_1 | REGULAR | 36.9 kB | 58 (since Fri Nov 21 2025) | UNIQUE | READY |
| createdAt_1 | REGULAR | 36.9 kB | 0 (since Fri Nov 21 2025) | TTL | READY |

- Data Example:

```
_id: ObjectId('692ff9313beeaba750061c70')
email : "pending1@example.com"
password : "$2b$10$0Sp0Q21MewUIJwfzpMRBpOsPrXApCrAWe5qsuwzK56EXugRIahfzK"
username : "newtrainer1"
▸ profile : Object
otp : "123456"
role : "user"
otpExpires : 2025-12-03T08:55:45.881+00:00
attempts : 1
recreation : 0
createdAt : 2025-12-03T08:45:45.881+00:00
__v : 0
updatedAt : 2025-12-03T08:47:45.885+00:00
```

# 5 User Interface and User Experience Design

## 5.1 Screen Diagram




**Screen List**

| Seq | Screen | Description |
|---|---|---|
| 1 | Login / Register | The entry point where users authenticate or create a new account to access the system. |
| 2 | Home Page (Social Feed) | The central dashboard displaying posts, navigation bars, and utility links (Map/Pokédex). |
| 3 | User Profile | Displays user information, avatar, followers/following counts, and their personal post history. |
| 4 | Interactive Map | The core feature allowing users to view game maps, zoom/pan, and track Pokémon locations. |
| 5 | Pokédex List | A searchable grid view of all Pokémon available in the selected game. |
| 6 | Pokémon Detail | A detailed view of a specific Pokémon's stats, type, weaknesses, and evolution chain. |

| | | |
|---|---|---|
| 7 | Edit Profile | The interface allowing the account owner to modify their avatar, display name, and bio information . |
| 8 | New Post | A form or overlay where users can draft content, upload images, and publish new posts to the social feed . |
| 9 | Comment Section | A detailed view of a specific post that allows users to read conversation threads and submit their own comments. |

## 5.2 Screen Specifications

### *5.2.1 Screen "Home Page (Social Feed)"*

**Presentation Format:**

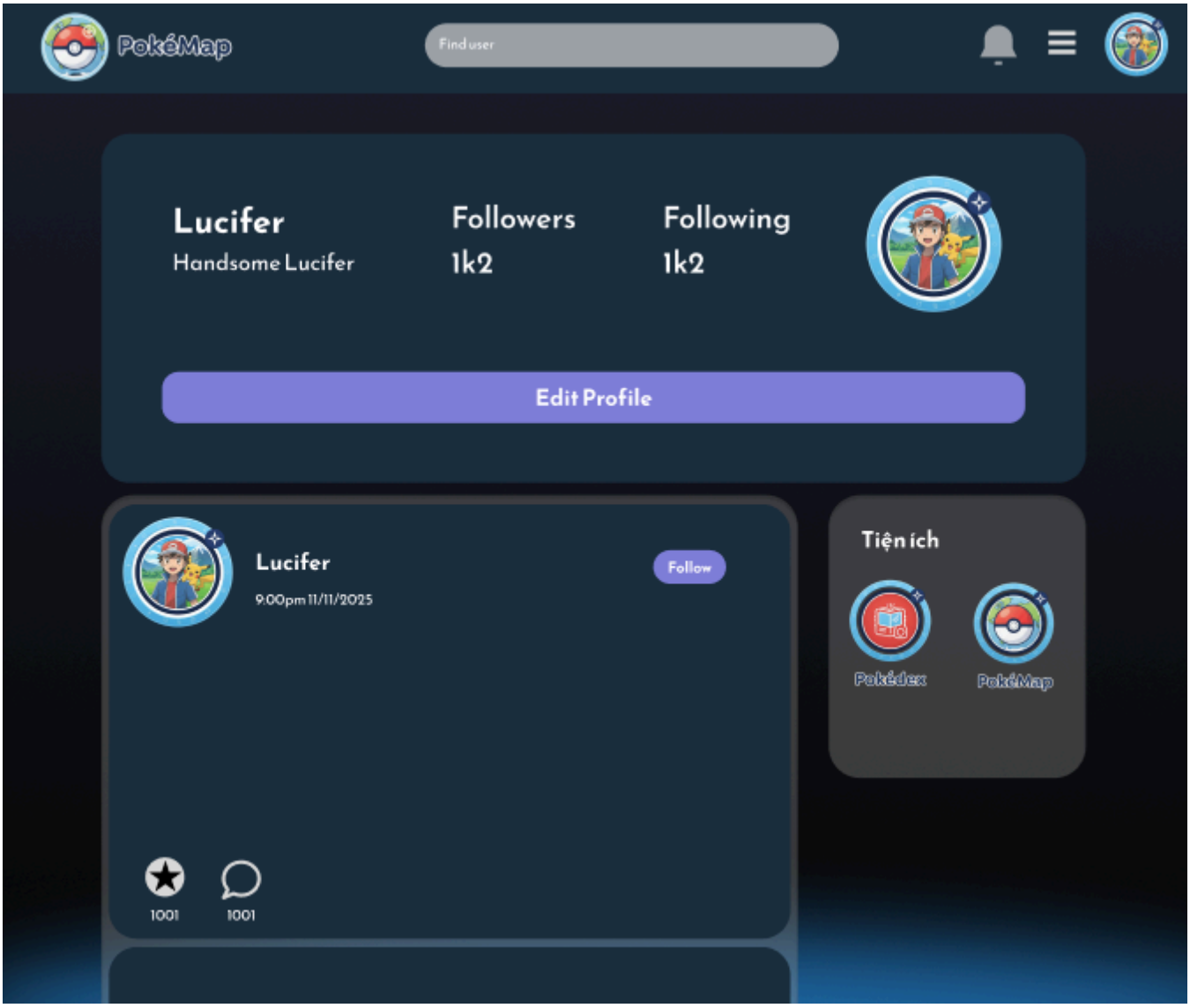
- Header: Contains the "PokéMap" logo, a Search Bar (TextBox) for finding users, and a Notification Bell (Button) .
- Sidebar (Right): A "Utilities" panel containing icons (Buttons) for the Pokédex and PokéMap systems .

- Main Content Area: A scrollable list (ListView) of User Posts. Each post displays the author's avatar, name, timestamp, post content (Text/Image), and interaction buttons .

**Event Handling:**

- Click Avatar: Triggers a transition to the User Profile screen corresponding to that user.
- Click "PokéMap" Icon: Switches the system to the Interactive Map screen.
- Click “PokéDex” Icon: Switches to the Pokemon list
- Click "Like" Button: Updates the local counter and sends a request to the database to register a "Like".
- Click "Comment" Button: Shows a form to post a comment.
- Search Input: Typing in the "Find User" TextBox queries the database and updates a dropdown list of matching users.

### *5.2.2 Screen "User Profile"*




**Presentation Format:**

- Profile Header: Displays the User's Avatar (Image), Username (Label), Bio (Label), and counts for Followers/Following .
- Action Controls:
  - Owner View: An "Edit Profile" Button.
  - Visitor View: A "Follow" Button.
  - Post History: A filtered list of posts created solely by this user.

**Event Handling:**

- Click "Edit Profile": Opens a modal or form allowing the user to update their Avatar, Name, and Tagline.
- Click "Follow": Toggles the follow status between the current user and the profile owner; updates the "Followers" count.
- Click "Return Home Page": Navigates back to the Home Page.

### *5.2.3 Screen "PokéMap"*

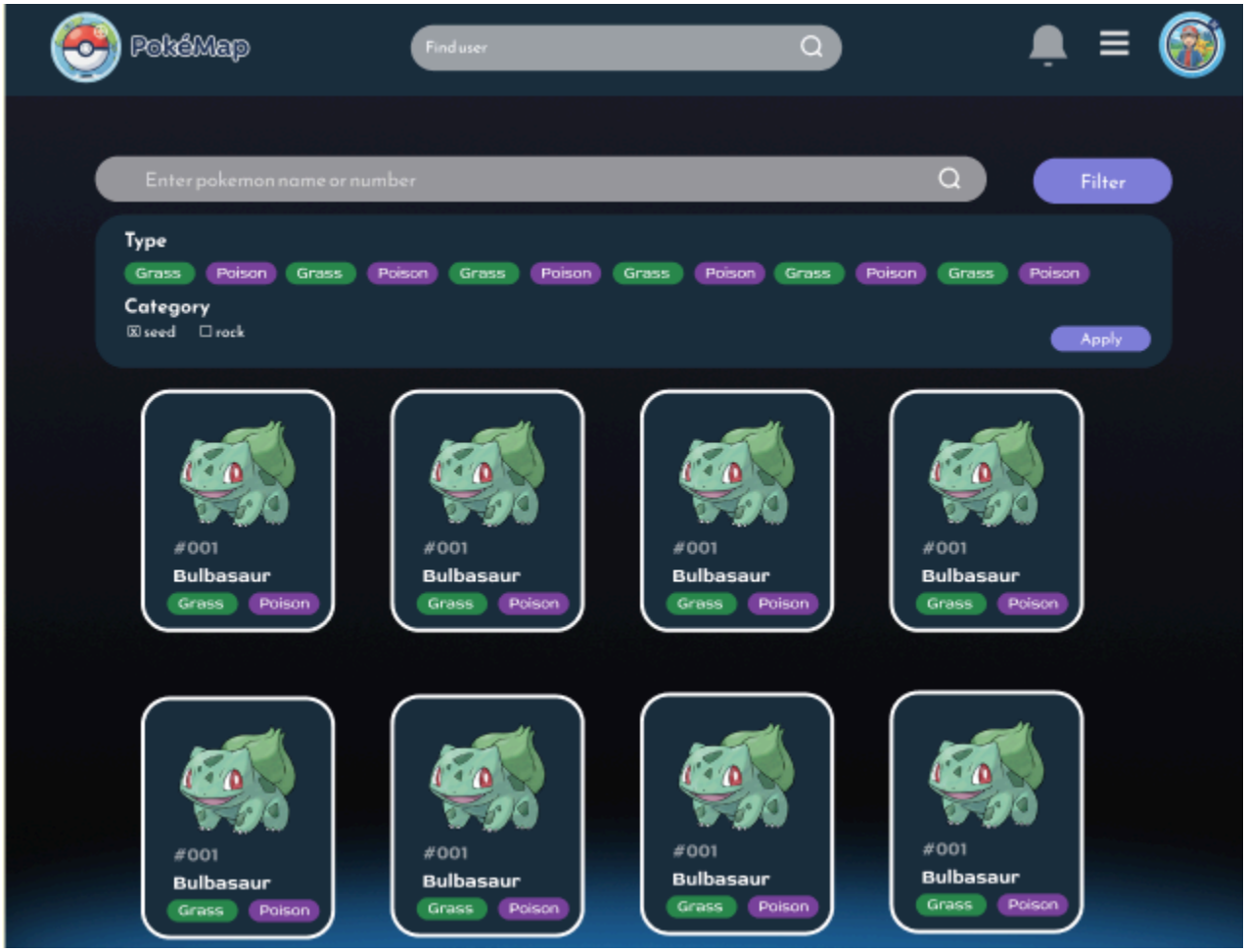
**Presentation Format:**

- Controls: A "Game Selector" (ComboBox) to choose the Pokémon game version and a "Region List".
- Viewport: A large zoomable canvas displaying the game map.
- Pins: Overlay icons representing Pokémon spawn locations.

**Event Handling:**

- Select Game: Reloads the map canvas with the image and data corresponding to the selected game.
- Click Pin: Marks the pin as "Found/Exhausted," changing its visual opacity to indicate the user has tracked it.
- Zoom/Pan: Updates the map viewport coordinates based on user input (mouse drag or pinch).

### *5.2.4 Screen "PokéDex"*



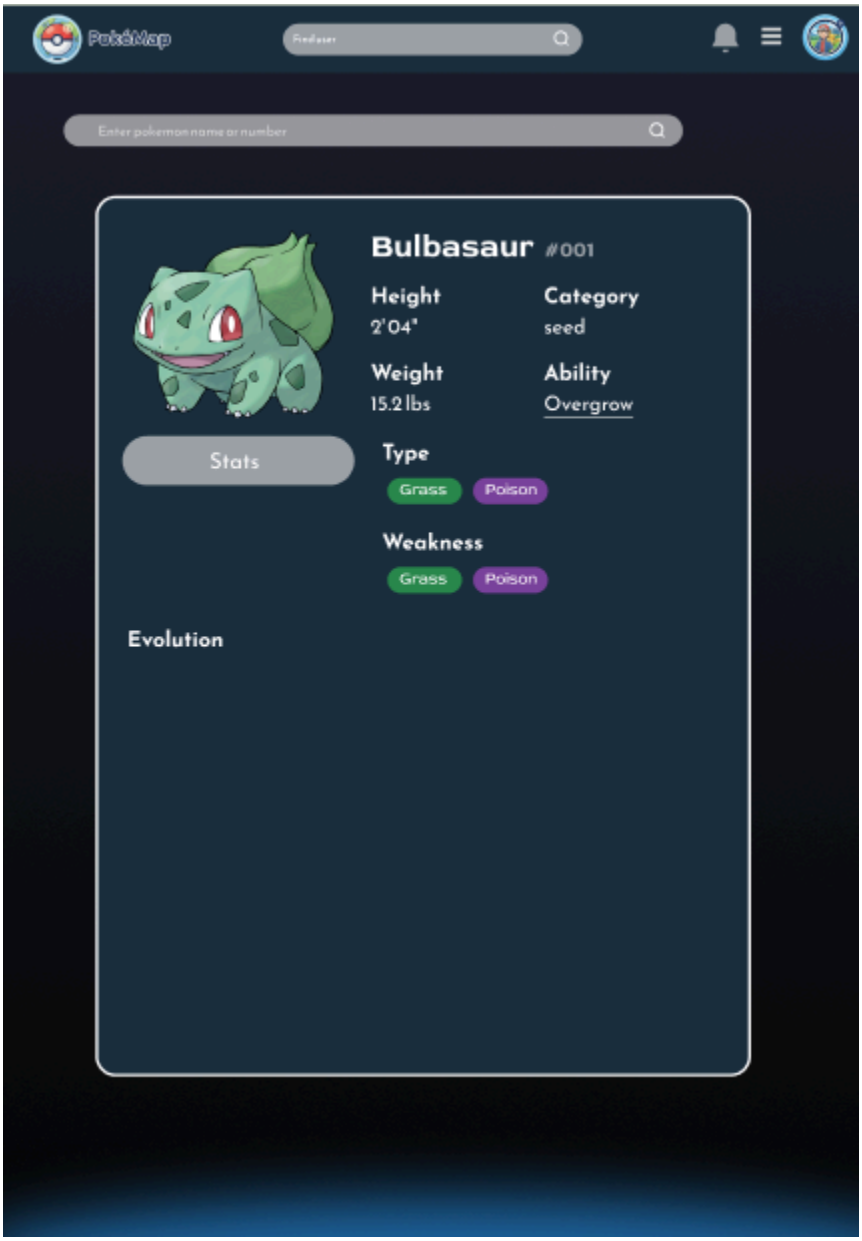

**Presentation Format:**

- Search/Filter: A Search Bar (TextBox) for Name/Number and a Filter button for Pokémon Type.
- Filter section: A section where the user clicks on an attribute to filter results.
- Grid View: A GridView displaying Pokémon cards. Each card shows the sprite, Number (#001), Name (Bulbasaur), and Type badges (Grass, Poison).

**Event Handling:**

- Enter Search Text: dynamically filters the GridView to show only Pokémon matching the input string.
- Click Pokémon Card: Navigates to the Pokémon Detail screen for the selected entity.

### *5.2.5 Screen "Pokémon Detail"*



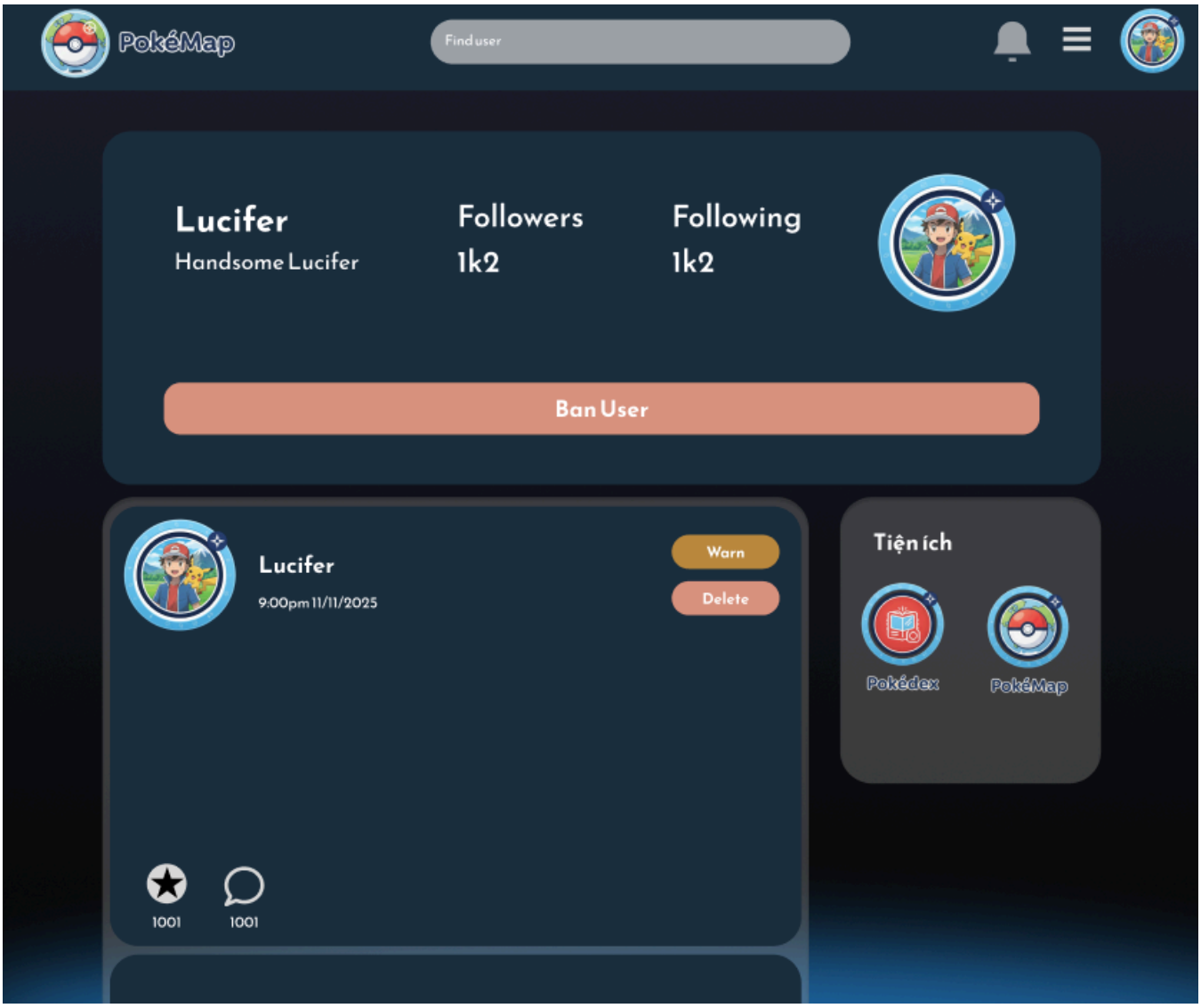

**Presentation Format:**

- Info Panel: Displays a high-resolution image of the Pokémon.
- Stats Table: Labels displaying Height, Weight, Category, and Ability .
- Type/Weakness: Visual badges indicating elemental attributes.
- Evolution Chain: A horizontal list showing the Pokémon's evolutionary stages.

**Event Handling:**

Click "Home Icon": Navigates back to Homepage

### *5.2.6 Screen "Moderator Control "*




.

**Presentation Format:**

- "Warn/Delete" button: Displays next to user's post.
- "Ban" button: Displays next to user's avatar.

**Event Handling:**

- Click "Ban User": Triggers a system action to change the user's status to 'banned' and restricts their access to the platform.
- Click "Warn": Sends a warning notification to the post owner regarding the specific content violation.
- Click "Delete"**:** Executes a command to remove the selected post from the database and immediately hides it from the public feed.